FOR LOOPS & NESTED LOGIC

Advanced Pattern Generation & Data Traversal

CodeHive Academy

Module 9: High-Efficiency Iteration

1. The 'for' Loop Mastery

A `for` loop is used when you know exactly how many times you want to repeat a block of code. It is the most common and robust way to iterate in C++.

The Syntax Anatomy

```
for (statement 1; statement 2; statement 3) { // code block to be executed }
```

- **Statement 1:** Runs once before the loop (e.g., `int i = 0;`).
- **Statement 2:** The condition. If true, the loop runs; if false, it exits.
- **Statement 3:** Runs after each cycle (e.g., `i++`).

2. Practical Example: Simple Counting

This implementation visualizes the counting process from 0 up to 4.

```
for (int i = 0; i < 5; i++) { cout << i << "\n"; }
```

The Logic Cycle:

- `int i = 0` sets the starting line.
- `i < 5` ensures the loop terminates before reaching 5.
- `i++` increments the value by 1 per cycle.

> If you wanted to count by twos, you would simply use `i += 2` in statement 3.

3. Iterating Through Arrays

As you build a platform like CodeHive or LeetCode, looping through data structures is a high-frequency skill.

```
string languages[] = {"C++", "Java", "Python"}; for (int i = 0; i < 3; i++)
{ cout << languages[i] << "\n"; }
```

Crucial Rule: C++ arrays are **0-indexed**. The loop counter `i` acts as the dynamic address to each element.

4. Nested Loop Structure

Nesting turns 1D logic into 2D capability. The program starts the outer loop, enters the inner loop, finishes the inner loop entirely, then returns to the outer loop for turn two.

```
// Outer loop for (int i = 1; i ⩽ 2; ++i) { // Inner loop for (int j = 1; j ⩽ 3; ++j) { cout << "Outer: " << i << ", Inner: " << j << "\n"; } }
```

Execution Breakdown:

1. Outer loop starts (i=1).
2. Inner loop runs 3 times (j=1, 2, 3).
3. Outer loop increments (i=2).
4. Inner loop runs 3 times again.

5. Generating Tables & Grids

Nested loops are the standard way to print data patterns like multiplication tables or coordinate maps for games.

```
for (int i = 1; i ⩽ 5; i++) { // Rows for (int j = 1; j ⩽ 5; j++) { // Columns cout << (i * j) << "\t"; // Product + Tab } cout << "\n"; // Row break }
```

> The `"\t"` escape sequence ensures the numbers align perfectly in columns.

6. Navigating 2D Arrays (Matrices)

On a high-level coding platform, searching through a 2D matrix is solved with nested logic.

```
int matrix[2][3] = { {1, 2, 3}, {4, 5, 6} }; for (int i = 0; i < 2; i++) {
for (int j = 0; j < 3; j++) { cout << matrix[i][j] << " "; } }
```

Note: The outer loop navigates Rows while the inner loop navigates Columns.

7. Real-Life Analogy: The Global Clock

To understand nesting, think of a digital timepiece. The **Hour** only increments after the **Minutes** have completed a full cycle.

```
for (int hr = 1; hr ⩽ 12; hr++) { for (int min = 0; min ⩽ 59; min++)
{ cout << hr << ":" << min << "\n"; } }
```

- **Outer Loop (Hours):** Runs 12 times.
- **Inner Loop (Minutes):** Runs 60 times per hour.
- **Total Hits:** 720 iterations.

8. Real-Life Analogy: Cinema Seating

Imagine a CodeHive theatre management system.

Outer Loop: Moves through each Row (A, B, C).

Inner Loop: Iterates through every Seat (1, 2, 3...).

For a worker to clean the theatre, they must touch every **inner seat** for every **outer row**.

9. Real-Life Analogy: Calendars

A standard month is a perfect 2D structure.

- **Outer Loop:** Iterates through the weeks (Week 1 to 4).
- **Inner Loop:** Iterates through 7 days of each week.

> This structure allows you to calculate statistical data, like "Total sales for Week 2."

10. Skill Lab: Building Shapes

A classic CodeHive interview question: Print a right-angled triangle using nested loops.

```
for (int i = 1; i ⩽ 5; i++) { for (int j = 1; j ⩽ i; j++) { cout << "*"; } cout << "\n"; }
```

Key Change: In the inner loop, notice `j <= i`. The inner loop's end-point depends on the outer loop's current progress!

11. Performance & Big O Notation

As a developer at CodeHive Academy, you must consider efficiency.

- **O(n):** A single for loop (Linear time).
- **O(n^2):** A nested loop (Quadratic time).

Double nested loops grow very fast. If n is 1,000, n^2 is 1,000,000 iterations. Always optimize your conditions to exit as early as possible.

12. Controlling Nested Loops

By default, `break` only exits the loop it is currently inside.

```
for( ... ) { for( ... ) { if(condition) break; // Exits inner loop only! } //
Outer loop continues here }
```

To exit both, you need a boolean flag or a `return` statement.

13. Master Practice Lab

Test your iteration skills with these CodeHive Academy assignments:

1. **FizzBuzz:** Loop 1 to 50. If divisible by 3, print "Fizz". By 5, "Buzz". Both, "FizzBuzz".
2. **Matrix Finder:** Populate a 4×4 matrix and find if the value '7' exists.
3. **Pyramid Maker:** Print a centered pyramid of stars using spaces and nesting.

14. Summary Checklist

Mastering for loops and nesting at CodeHive unlocks the path to complex data handling.

- ✅ Use `for` when the iteration count is known.
- ✅ Master the 0-index for array traversal.
- ✅ Use nesting for 2D structures (grids/matrices).
- ✅ Remember: Inner loops complete fully for every ONE outer step.
- ✅ Be mindful of performance (n^2 traps).

Code. Learn. Repeat.